

**A
Project
Report On
" ParkSphere Connect - Smart Parking Management
System"**

**(SEP 728 – Internet of Things and Industrial Internet of
Things Systems)**

Prepared by
Drashti Vaghasiya
Hetvi Vaghasiya
Kaushal Rathod
Mithil Mangukia

Under the guidance of
Prof. Pedro Tondo

Submitted at

**W Booth School of Engineering Practice and
Technology McMaster University
1280 Main St W, Hamilton, ON, CA
January 2026 - April 2026**

ABSTRACT

This project presents **ParkSphere Connect**, an IoT-based smart parking system designed to improve parking efficiency through real-time monitoring and automation. The system utilizes a Raspberry Pi as the central controller and integrates multiple components including an ultrasonic sensor for distance measurement, a servo motor for gate control, an LED for status indication, and a buzzer for proximity alerts.

The ultrasonic sensor continuously measures the distance to detect vehicle presence and determine parking slot availability. Based on this data, the system automatically controls the gate mechanism and provides visual and audio feedback. Additionally, PubNub is used for real-time cloud communication, enabling remote monitoring and control of the system. This solution offers a cost-effective and scalable approach to smart parking, reducing manual intervention and improving user experience.

The application for ParkSphere Connect is a real-time monitoring dashboard built with the React.js framework, designed to bridge the gap between physical sensor data and a user-friendly digital interface. By subscribing to the PubNub cloud stream, the software instantly processes incoming JSON packets from the Raspberry Pi to update a dynamic parking map, toggling slot availability between green and red based on live ultrasonic distance readings. The interface provides comprehensive telemetry, including the exact distance of a vehicle from the sensor and the current mechanical state of the servo-controlled gate, while also maintaining a persistent history log of all recent parking activities. This integrated software solution transforms raw hardware signals into an intuitive, data-driven experience that allows both drivers and administrators to manage the parking facility with high precision and remote accessibility.

Table of Contents

ABSTRACT	II
TABLE OF CONTENTS	III
LIST OF FIGURES	IV
CHAPTER 1: INTRODUCTION	5
1.1 BACKGROUND	5
1.2 OBJECTIVE	5
1.3 SCOPE	6
CHAPTER 2: SYSTEM OVERVIEW	7
2.1 OVERALL SYSTEM ARCHITECTURE	7
2.2 DATA ACQUISITION WORKFLOW	8
2.3 REAL-TIME PROCESSING AND MONITORING	9
2.4 SYSTEM DEPLOYMENT AND SCALABILITY	9
CHAPTER 3: HARDWARE REQUIREMENTS	11
3.1 RASPBERRY PI 4	11
3.2 ULTRASONIC SENSOR	12
3.3 IR SENSOR	13
3.4 SERVO MOTOR	14
3.5 LED AND BUZZERS	15
CHAPTER 4: SOFTWARE REQUIREMENTS	16
4.1 PYTHON	16
4.2 PUBNUB	17
4.3 WEB APPLICATION	17
CHAPTER 5: SYSTEM ARCHITECTURE	19
5.1 ARCHITECTURE	19
5.2 CIRCUIT DIAGRAM	20
CHAPTER 6: RESULTS AND VISUALIZATION	23
6.1 RESULT DISCUSSION	23
CHAPTER 7: LIMITATIONS AND FUTURE SCOPE	24
7.1 LIMITATIONS	24
7.2 FUTURE SCOPE	24
CHAPTER 8: REFERENCES	26

Table of Figures

1. System Architecture	7
2. Raspberry Pi	10
3. Ultrasonic Sensor	11
4. IR Sensor	12
5. Servo Motor	13
6. LED and Buzzer	14
7. Circuit Diagram	22

CHAPTER 1: INTRODUCTION

1.1 Background

In today's rapidly growing urban environments, parking management has become a major challenge due to the increasing number of vehicles. In places like residential complexes, shopping malls, offices, and universities, drivers often spend a significant amount of time searching for available parking spaces. This not only wastes time but also leads to increased fuel consumption, traffic congestion, and frustration among users. Traditional parking systems are mostly manual or lack real-time monitoring, which makes them inefficient and unreliable. There is usually no proper way to know whether a parking slot is available before reaching the location, and this results in unnecessary circulation of vehicles within parking areas.

With the advancement of Internet of Things (IoT) technology, it has become possible to develop smart systems that can automate and optimize such processes. IoT enables communication between physical devices like sensors and controllers through the internet, allowing real-time data collection and monitoring. By using sensors to detect parking availability and connecting them to a cloud-based system, users can get live updates about parking spaces. The **ParkSphere Connect** project is based on this concept, where smart sensing and cloud communication are combined to create an efficient parking monitoring and control system.

1.2 Objective

The main objective of this project is to design and develop a smart parking system that can automatically detect the availability of parking slots and control access to the parking area. The system aims to reduce the time and effort required by drivers to find a parking space by providing real-time information. It also focuses on automating the entry process using sensors and actuators, which reduces the need for human intervention.

Another important objective of the project is to integrate cloud communication using PubNub so that the parking status can be monitored remotely. This allows real-time updates and also enables remote control features such as opening or closing the gate manually through commands. The system is also designed to provide safety features like buzzer alerts when a vehicle is too close and visual indicators using LEDs to show slot availability.

Overall, the project aims to improve efficiency, reduce congestion, and demonstrate how IoT can be used in real-life applications like smart parking systems.

1.3 Scope

The scope of this project is focused on developing a functional prototype of a smart parking system using Raspberry Pi and various sensors. The system is designed to monitor a single parking slot and control the gate based on its availability. Even though it is a small-scale implementation, it demonstrates the core concept of a smart parking system that can be expanded in the future.

This project can be applied in different environments such as residential parking areas, office buildings, shopping centers, and university campuses. It can also be extended to support multiple parking slots by adding more sensors and integrating them into a centralized system. Furthermore, the system can be enhanced by adding mobile applications, reservation features, and payment integration, making it suitable for large-scale smart city applications.

CHAPTER 2: SYSTEM OVERVIEW

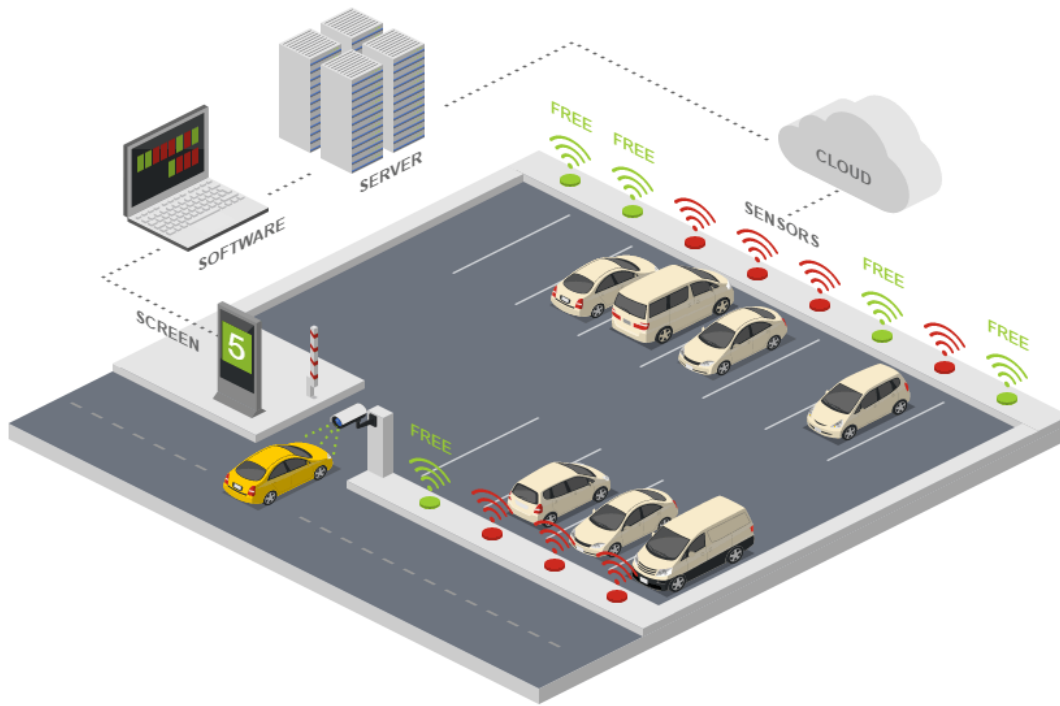
2.1 Overall System Architecture

The ParkSphere Connect system is designed using a layered architecture approach, which helps in organizing the system into different functional parts for better understanding and performance. The architecture mainly consists of three layers: sensing layer, processing layer, and communication layer. Each layer has its own role, and together they work to provide a complete smart parking solution. This structured design makes the system more efficient, scalable, and easy to manage, especially when extending it to larger applications in the future.

The sensing layer is responsible for collecting real-time data from the physical environment. In this system, an ultrasonic sensor is used to detect the availability of the parking slot by measuring the distance between the sensor and the object. If the distance is above a certain threshold, the slot is considered free; otherwise, it is occupied. Along with this, an IR sensor is placed at the gate to detect the presence of a vehicle. This combination of sensors ensures that both parking availability and vehicle entry conditions are monitored accurately and continuously.

The processing layer is handled by the Raspberry Pi, which acts as the central controller of the entire system. It continuously receives input data from the sensors and processes it using a Python program. Based on predefined conditions and logic, the Raspberry Pi makes decisions such as whether to open or close the gate. For instance, if a vehicle is detected at the entrance and the parking slot is free, the system automatically triggers the servo motor to open the gate. It also controls other components like LEDs and buzzers to provide visual and audio feedback, making the system more interactive and responsive.

The communication layer is implemented using PubNub, a cloud-based real-time data streaming platform. This layer enables the system to send parking status updates, such as slot availability, distance values, and gate status, to the cloud. These updates can be accessed through a dashboard for monitoring purposes. Additionally, PubNub allows the system to receive remote commands, such as forcing the gate to open or close, providing flexibility and remote control capabilities. This integration of cloud communication ensures that the system is not limited to local operation but can be monitored and controlled from anywhere in real time.



1. System Architecture

2.2 Data Acquisition Workflow

The data acquisition workflow in the ParkSphere Connect system is designed to continuously collect, process, and transmit information related to parking slot availability and vehicle detection. The process begins with the ultrasonic sensor, which regularly measures the distance between itself and any object placed in front of it. This measurement helps determine whether a parking slot is occupied or free. The sensor sends ultrasonic waves and calculates the time taken for the waves to return after reflecting from an object. Based on this time, the system computes the distance and compares it with a predefined threshold to classify the slot status.

At the same time, the IR sensor plays an important role in detecting the presence of a vehicle at the parking gate. When a vehicle approaches the entry point, the IR sensor detects it and sends a signal to the Raspberry Pi. The Raspberry Pi then combines the data from both sensors to make decisions. For example, if a vehicle is detected at the gate and the parking slot is available, the system proceeds to open the gate using the servo motor. If the slot is occupied, the gate remains closed, preventing unnecessary entry and maintaining proper parking management.

Once the data is processed and decisions are made, the system sends all relevant information to the cloud using PubNub. This includes parameters such as slot status (free or occupied), measured distance, and gate condition. The data is transmitted in real time, allowing it to be displayed on a dashboard for monitoring purposes. This continuous cycle of sensing, processing, and communication ensures that the system always provides up-to-date information and responds quickly to changes, making the parking system efficient and reliable.

2.3 Real-Time Processing and Monitoring

Real-time processing is a key feature of the ParkSphere Connect system. The Raspberry Pi runs a Python program that continuously executes in a loop, reading sensor values and making decisions instantly. The ultrasonic sensor data is processed to calculate the distance, and based on this value, the system determines the availability of the parking slot.

The system also provides real-time feedback through hardware components. An LED is used to indicate whether the slot is free or occupied, while a buzzer is activated if a vehicle is detected too close to the sensor, providing an alert. The servo motor is controlled to open or close the gate depending on the situation.

All this information is simultaneously sent to the PubNub cloud, where it can be visualized on a dashboard. This ensures that users can monitor the system in real time and take actions if required.

2.4 System Deployment and Scalability

The deployment of the ParkSphere Connect system is designed in a way that it can be easily implemented in real-world environments with minimal setup requirements. In its current form, the system is deployed as a prototype using a single Raspberry Pi connected with sensors and actuators to monitor one parking slot. The hardware components are arranged in a compact setup where the ultrasonic sensor is positioned to detect the parking space, and the IR sensor is installed at the entry point to detect incoming vehicles. The Raspberry Pi acts as the central controller, running the Python program continuously and managing all operations. This setup makes the system suitable for demonstration purposes as well as small-scale applications such as individual parking spots in residential areas or small offices.

From a deployment perspective, the system is flexible and does not require complex infrastructure. It can be installed in existing parking areas without major modifications, making it cost-effective and easy to maintain. The use of wireless communication through PubNub allows the system to operate without heavy wiring for data transfer, which further simplifies installation. Additionally, since the system is based on IoT principles, it can be connected to cloud platforms, enabling remote monitoring and control through dashboards or web interfaces. This makes it highly useful for environments where real-time parking information is needed.

In terms of scalability, the current system can be extended to support multiple parking slots by adding more ultrasonic sensors and integrating them with the same Raspberry Pi or multiple controllers. Each slot can be monitored individually, and the data can be aggregated and sent to a

centralized cloud system. This allows the system to handle larger parking areas such as malls, corporate buildings, or university campuses. By using cloud-based communication, all parking data from multiple slots can be visualized in a single dashboard, providing a complete overview of parking availability.

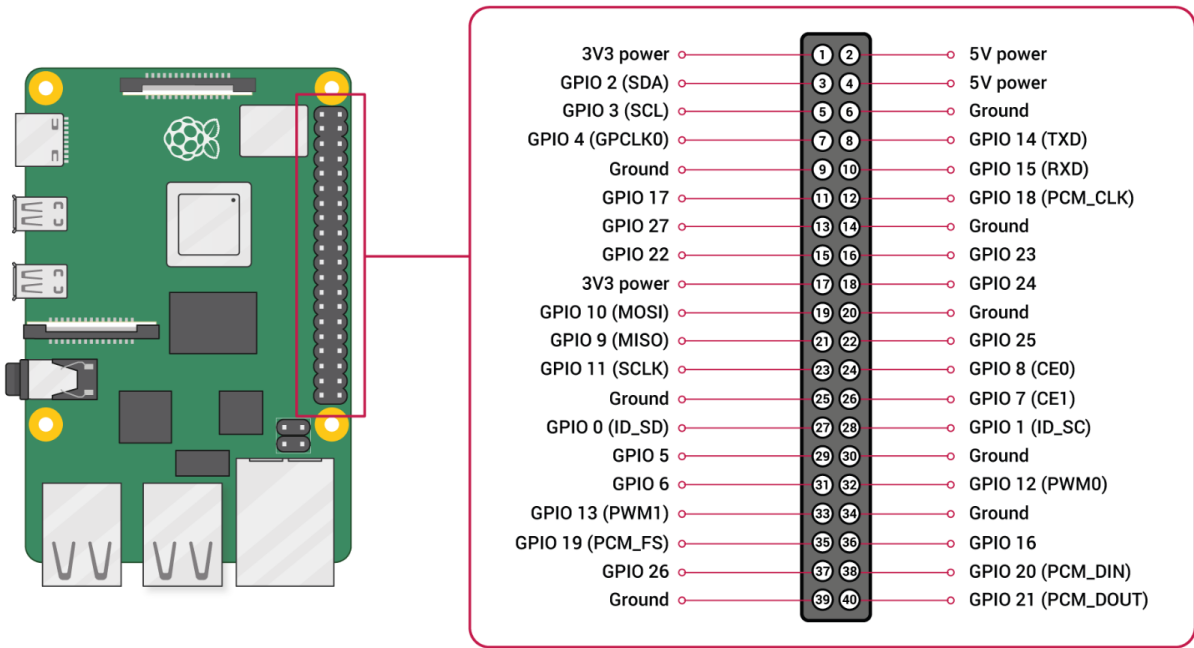
Furthermore, the system can be enhanced for large-scale smart city applications by integrating advanced features such as mobile applications for users, online reservation systems, and automated payment options. Machine learning algorithms can also be added in the future to predict parking availability based on historical data. With these improvements, the ParkSphere Connect system has the potential to evolve from a simple prototype into a fully functional smart parking solution capable of handling complex, real-world scenarios efficiently.

CHAPTER 3: HARDWARE REQUIREMENTS

3.1 Raspberry Pi 4

The Raspberry Pi 4 acts as the central processing unit of the entire system, coordinating all hardware and software operations efficiently. It continuously collects data from connected sensors, such as ultrasonic or IR sensors, and processes this information to determine the status of parking slots. Based on the processed data, it sends appropriate signals to output components like the servo motor for gate control, LEDs for visual indication, and a buzzer for alerts. Its ability to run a full operating system allows the system to handle multiple tasks simultaneously, making the overall system more responsive and reliable.

In addition to hardware control, the Raspberry Pi 4 also plays an important role in communication and system integration. It can connect to the internet using Wi-Fi or Ethernet, enabling real-time data transmission to cloud platforms or mobile applications for monitoring and management. This feature makes the system scalable and suitable for smart city applications. The Raspberry Pi is selected not only for its affordability and compact design but also for its flexibility, strong community support, and compatibility with various programming languages, which simplifies development and future upgrades.



2. Raspberry Pi

3.2 Ultrasonic Sensor

The ultrasonic sensor serves as the primary "eye" of the smart parking system, utilizing high-frequency sound waves to monitor slot occupancy. Operating on the principle of echolocation, the sensor emits an ultrasonic pulse from its trigger pin, which travels through the air until it encounters an obstruction, such as a vehicle. Once the pulse hits the object, it bounces back as an echo and is captured by the receiver pin. By measuring the precise time interval between the transmission and reception—often referred to as the Time of Flight (ToF)—the system can accurately determine the distance of the object from the sensor.

To translate raw time data into a meaningful distance measurement, the system applies a specific mathematical formula based on the speed of sound. Since sound travels at approximately 343 meters per second in dry air at 20°C, the distance is calculated by multiplying the time by the speed and dividing by two (to account for the round-trip journey). In a parking context, the system is programmed with a threshold distance: if the detected distance is significantly shorter than the height of the ceiling or the floor of an empty stall, the system identifies the presence of a vehicle. This logic allows the microcontroller to toggle the status of the parking slot between "Available" and "Occupied" in real-time.

Integrating ultrasonic technology offers several advantages for automated parking management. Unlike optical sensors or cameras, ultrasonic waves are largely unaffected by ambient light conditions, meaning they function perfectly in dimly lit underground garages or under bright direct sunlight. Furthermore, they are highly resistant to interference from dust, smoke, or light rain, ensuring consistent performance in diverse environments. Their low power consumption and cost-effectiveness make them an ideal solution for scaling a smart parking network across hundreds of individual stalls, providing a reliable and maintenance-friendly method for reducing traffic congestion and improving the driver experience.



3. Ultrasonic Sensor

3.3 IR Sensor

The Infrared (IR) sensor operates as a proximity-based detection tool within the smart parking system, relying on the emission and reception of light signals. Each sensor module consists of an IR LED (Transmitter) and a Photodiode (Receiver). The transmitter emits a beam of infrared light that is invisible to the human eye. When a vehicle enters the parking bay, it reflects this light back toward the sensor. The photodiode detects the intensity of the reflected light and converts it into an electrical signal. This "reflection-based" logic allows the system to identify the presence of an object within a short range without requiring physical contact.

In a smart parking architecture, IR sensors are typically positioned at the entry and exit points or at the front of each parking stall to provide real-time occupancy data. Because IR sensors respond almost instantaneously to changes in their field of view, they are excellent for triggering automated gates or updating LED indicators (turning to Green) the moment a car pulls into a spot. While they are more sensitive to direct sunlight than ultrasonic sensors, their compact size and low cost make them a highly efficient choice for indoor garage applications where precise, localized detection is required to manage traffic flow and guide drivers to vacant spaces.



4. Infrared Sensor

3.4 Servo Motor

The servo motor acts as the physical actuator for the automated barrier gate in this smart parking system. Unlike standard DC motors, the servo is designed for precise angular positioning, which is essential for the 90-degree movement required to raise and lower a gate. In this project, the motor is interfaced with GPIO 12 of the Raspberry Pi and controlled using Pulse Width Modulation (PWM) at a frequency of 50Hz. By varying the Duty Cycle—specifically set to 7% for opening and 2% for closing—the Raspberry Pi sends high-speed electrical pulses that tell the motor exactly which position to hold, ensuring the gate operates smoothly and stays locked in place when idle.

Within the system architecture, the servo motor is the final execution point for both local and cloud-based logic. It remains in a "closed" state by default, only triggering an `open_gate()` function when the IR sensor detects an approaching vehicle and the ultrasonic sensor confirms a slot is available (distance > 10cm). This prevents entry when the lot is full. Additionally, the integration with PubNub allows for a "Remote Override" feature; the servo can respond to `force_open` or `force_close` commands sent from a remote dashboard. This dual-layer control—combining local sensor data with real-time cloud commands—provides a robust and secure entry management solution.



5. Servo Motor

3.5 LED and Buzzers

The LED, connected to GPIO 18, serves as the system's primary visual interface for communicating slot occupancy to the user. Managed by the Raspberry Pi's logic, the LED is programmed to illuminate only when the ultrasonic sensor returns a distance greater than 10 cm, signaling that the parking space is "Free." This automated signaling eliminates the need for manual inspection of parking bays, allowing drivers to identify available spots at a glance, which significantly streamlines traffic flow within the parking facility.

The Buzzer, interfaced via GPIO 20, provides an essential audible safety layer by alerting drivers to potential collisions during the parking process. The system monitors the ultrasonic distance readings in real-time and triggers the buzzer to emit a continuous warning tone if a vehicle moves within a 5 cm threshold of the sensor. By providing this immediate "Too Close" feedback, the buzzer compensates for driver blind spots and ensures that the smart parking system is not only an automated management tool but also a safety-enhancing application.



6. Buzzer and LED

CHAPTER 4: SOFTWARE REQUIREMENTS

4.1 Python

Python is used as the primary programming language for implementing the system logic. It is easy to use and provides libraries for hardware control and communication. The Python script continuously runs on the Raspberry Pi and handles all operations, including sensor reading, decision-making, and cloud communication.

The RPi.GPIO library is the essential interface between the Python code and the Raspberry Pi's physical hardware. It is used to configure the pin numbering system (BCM) and define whether each connected component acts as an input or an output. For example, Our code uses GPIO.setup() to establish the Trig and Echo pins for the ultrasonic sensor, the IR pin for entry detection, and the PWM channel for the servo motor. Without this library, the script would have no way to send electrical signals to the LED and buzzer or read the digital pulses coming from the sensors.

The PubNub library facilitates the "Smart" aspect of the project by providing real-time cloud connectivity and IoT messaging. By initializing the PubNub and PNConfiguration classes with unique keys, the script can publish the parking status (Distance, Slot Status, and Gate Status) to a remote channel named "group_7." Furthermore, the SubscribeCallback class allows the system to listen for incoming "Remote Override" commands. This enables the parking gate to be controlled from anywhere in the world, transforming a local hardware setup into a fully integrated cloud application.

The Time library is used to manage the temporal logic and synchronization of the hardware components. In your script, it is most critical for calculating the ultrasonic distance; by capturing time.time() at the exact moment a pulse is sent and received, the code can measure the "Time of Flight" with microsecond precision. Additionally, time.sleep() is used to provide the necessary delays for the servo motor to reach its position and to prevent the main loop from overwhelming the CPU, ensuring the system runs smoothly and reliably over long periods.

4.2 Pubnub

- **Cloud-Based Real-Time Connectivity:** The PubNub SDK serves as the communication backbone of the smart parking system, enabling seamless data exchange between the local Raspberry Pi hardware and the cloud. By utilizing a Publish/Subscribe (Pub/Sub) architecture, the system can instantly transmit occupancy data, sensor readings, and gate statuses to a global network. In your project, the `pubnub.publish()` function broadcasts a JSON payload containing the slot status and distance to the "group_7" channel. This eliminates the need for complex server configurations or port forwarding, allowing the parking data to be monitored in real-time from any internet-connected device or dashboard.
- **Remote Command Execution and IoT Control:** Beyond simple data logging, PubNub enables bidirectional interaction through its Listener mechanism. By implementing the `SubscribeCallback` class, the Raspberry Pi remains in a constant state of "listening" for incoming messages from the cloud. This allows for the Remote Override functionality seen in your code, where specific commands like `force_open` or `force_close` can be sent to the system to control the servo motor manually. This integration transforms the project from a standalone automated gate into a sophisticated IoT (Internet of Things) solution, providing administrators with total oversight and the ability to intervene in the parking management process remotely.

4.3 Web Application

- **Application Architecture**
The user-facing side of the Smart Parking System is a modern web application built using the React.js framework. This choice ensures a responsive, component-based architecture that can update in real-time without refreshing the page. The application acts as a subscriber to the PubNub "group_7" channel, instantly consuming the JSON data packets sent by the Raspberry Pi. By leveraging React's state management, the UI reflects the live physical state of the parking garage, transforming raw sensor data into an intuitive graphical interface for both parking attendants and drivers.
- **Real-Time Monitoring and Occupancy Logic**
The core functionality of the dashboard is the Live Parking Slot Availability tracker. Using the distance data provided by the ultrasonic sensor, the app visually toggles the slot status between "Available" (Green) and "Occupied" (Red). Additionally, the UI displays the Exact Distance in centimeters and the current Gate Status (Open, Closed, or Idle). This level of detail allows users to see not just if a spot is taken, but how a vehicle is positioned, while the gate status indicator provides confirmation that the entry system is functioning correctly according to the IR sensor logic.
- **Data History and Activity Logging**
Beyond real-time tracking, the application features a Recent Activity and History module. This section logs every major event—such as a car entering the gate, a slot being vacated, or a "Too Close" buzzer alert—with a precise timestamp. By maintaining a scrollable

history of these activities, the system provides valuable analytics on parking patterns and peak usage times. This historical oversight is crucial for facility management, allowing for better resource allocation and providing a clear audit trail of all automated and remote-override actions performed by the system.

CHAPTER 5: SYSTEM ARCHITECTURE

5.1 Architecture

The architecture of this smart parking system is built around the Raspberry Pi, which serves as the central processing unit (CPU). The system utilizes the Broadcom (BCM) pin numbering convention to establish a reliable interface between the Python script and the physical hardware. By defining specific pins for inputs (sensors) and outputs (actuators), the Raspberry Pi acts as a decision-making hub that interprets environmental data and executes mechanical responses. This centralized control ensures that all components operate in a synchronized manner, allowing the software logic to manage power distribution and signal processing across the entire circuit.

At the heart of the "sensing layer" is the HC-SR04 Ultrasonic Sensor, which is responsible for monitoring the actual parking stall. The sensor is wired to GPIO 23 (Trigger) and GPIO 24 (Echo). The architecture relies on the "Time of Flight" principle, where the Raspberry Pi sends a 10 μ s pulse to the trigger pin and calculates the duration until the echo pin receives the reflected signal. This high-precision timing is handled by the Python time library, enabling the system to determine if a slot is "Occupied" or "Free" based on whether the detected distance is above or below a 10 cm threshold.

To manage the entry point of the parking facility, the architecture incorporates an Infrared (IR) Sensor connected to GPIO 17. This sensor acts as a digital trigger that detects the physical presence of a vehicle at the barrier. Unlike the ultrasonic sensor, which provides a variable distance, the IR sensor provides a binary signal (Low/High). When a car blocks the infrared beam, the signal drops to 0, notifying the Raspberry Pi to initiate the entry sequence. This dual-sensor approach ensures that the gate only attempts to open when a vehicle is physically present at the entrance and a space is confirmed available.

The mechanical "execution layer" consists of a Servo Motor connected to GPIO 12, which functions as the physical barrier gate. The architecture uses Pulse Width Modulation (PWM) to control the motor's position with high accuracy. By operating at a frequency of 50Hz, the Raspberry Pi can command the servo to rotate to a "7% duty cycle" to lift the gate or a "2% duty cycle" to close it. This component provides the necessary physical security for the system, ensuring that entry is only granted after the software logic validates that the lot is not at full capacity.

For real-time interaction with the driver, the architecture includes both visual and audible feedback loops. An LED, connected to GPIO 18, serves as a visual status indicator that remains lit when a parking slot is vacant. Simultaneously, an Active Buzzer is wired to GPIO 20 as a safety proximity alarm. If the ultrasonic sensor detects that a driver is parking too close to the wall or sensor (less than 5 cm), the Pi triggers the buzzer to provide an immediate audible warning. These components ensure that the system is user-friendly and helps prevent minor collisions during parking maneuvers.

The final layer of the architecture is the Cloud Messaging Layer, powered by the PubNub API. This allows the local hardware to communicate with an external network via the internet. Every second, the system packages the current distance, slot status, and gate position into a JSON message and publishes it to the "group_7" channel. Furthermore, the architecture supports bidirectional communication, meaning a remote administrator can send override commands back to the Raspberry Pi to force the gate open or closed. This transforms the setup from a localized circuit into a fully connected IoT smart city application.

5.2 Circuit Diagram

5.2.1 The Power and Ground Rails

The foundation of the circuit is the power distribution from the Raspberry Pi. The 5V pin (usually Pin 2 or 4) provides the "High" voltage needed for the Ultrasonic Sensor, Servo Motor, and Buzzer, as these components require more power to operate mechanical parts or sound waves. The 3.3V pin is used for lower-power components like the IR Sensor and LED. Most importantly, all components are connected to a common GND (Ground) rail on the breadboard to complete the electrical circuit and prevent signal noise.

5.2.2 Ultrasonic Sensor (HC-SR04) Connections

The ultrasonic sensor requires four connections. The VCC goes to 5V and GND goes to Ground. The logic happens at GPIO 23 (Trig) and GPIO 24 (Echo). When your code sets TRIG to True, it sends a signal to the sensor to fire a sound wave. When that wave bounces back, the sensor sends a signal to the ECHO pin, which the Raspberry Pi "listens" for to stop the timer.

- Trig: GPIO 23 (Output)
- Echo: GPIO 24 (Input)

5.2.3 Barrier Control (Servo Motor)

The Servo Motor is a three-wire component. The Red wire connects to 5V, the Brown/Black wire to GND, and the Orange/Yellow control wire connects to GPIO 12. In the circuit, GPIO 12 sends the PWM (Pulse Width Modulation) signal. Even though the power comes from the 5V rail, the "instructions" on how far to rotate (7% or 2% duty cycle) come exclusively from this single GPIO pin.

- Control Wire: GPIO 12 (PWM Output)

5.2.4 Entry Detection (IR Sensor)

The IR Sensor is a simple digital input. It has three pins: VCC (3.3V or 5V), GND, and OUT. The OUT pin is connected to GPIO 17. In your circuit, this pin is constantly monitored. When a car is in front of the sensor, the "OUT" signal drops to 0 (Low), which your code interprets as `ir_val == 0`, triggering the logic to check if the parking slot is free.

- Out: GPIO 17 (Input)

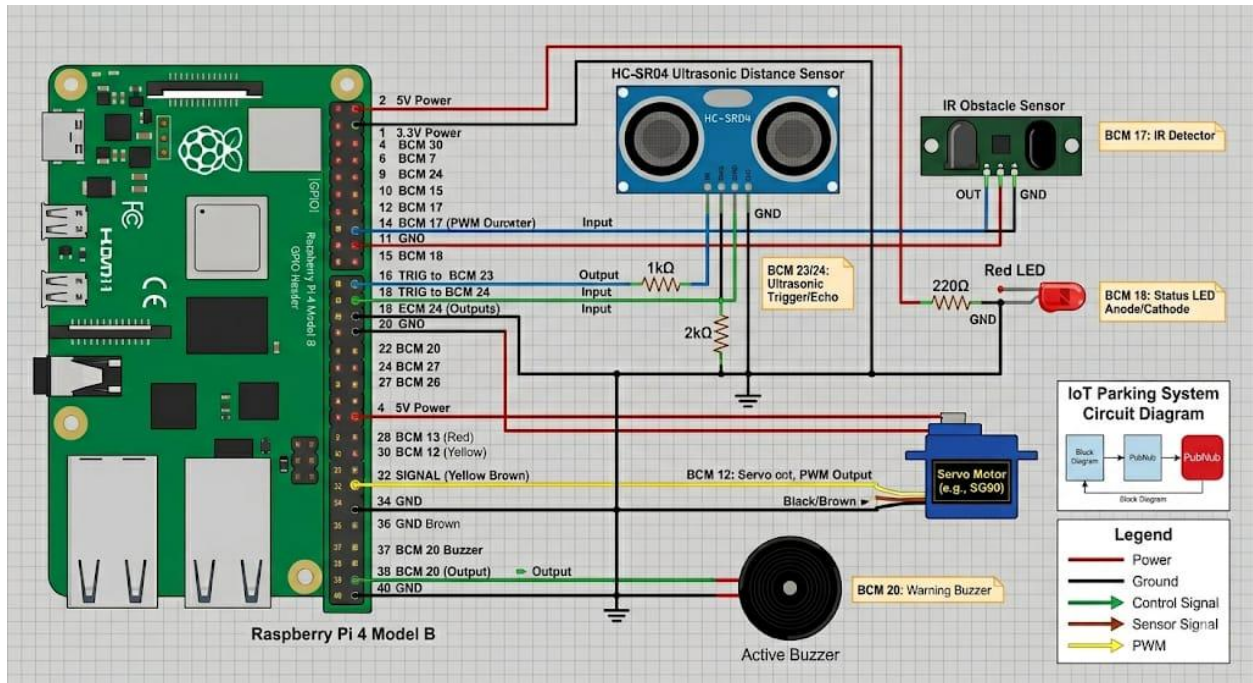
5.2.5 Visual and Audible Feedback (LED & Buzzer)

These are the simplest parts of the circuit diagram but provide the most user interaction.

- LED: The positive leg (Anode) is connected to GPIO 18 (usually with resistor to prevent burning out the LED), and the negative leg connects to Ground. It turns on when the distance is > 10 cm.
- Buzzer: The positive terminal is connected to GPIO 20 and the negative to Ground. It acts as an "Active" component, meaning as soon as GPIO 20 is set to True (when distance < 5 cm), it starts beeping.

5.2.6 The Logic Bridge (Code to Hardware)

In a formal circuit diagram, the Raspberry Pi sits in the center. The "Architecture" described is represented here by lines (wires) connecting the Pi's GPIO pins to the breadboard. The code acts as the "Switchboard Operator," deciding which pins to send voltage to (Outputs like LED/Buzzer/Servo) and which pins to read voltage from (Inputs like IR/Ultrasonic). This creates a closed-loop system where the physical environment (distance/car presence) directly dictates the mechanical response (gate opening).



7. Circuit Diagram

CHAPTER 6: RESULTS AND VISUALIZATION

6.1 Result Discussion

The proposed ParkSphere Connect Smart Parking System was successfully designed, implemented, and tested under various operating conditions to evaluate its performance and reliability. The results demonstrate that the system functions effectively and fulfills its intended objectives. The ultrasonic sensor played a key role in detecting the availability of parking slots. During testing, it was able to accurately measure the distance between the sensor and the vehicle, allowing the system to determine whether a slot was occupied or vacant. The readings were consistent in most cases, ensuring reliable parking status detection.

Similarly, the IR sensor installed at the entry point effectively detected the presence of vehicles approaching the gate. This input enabled the system to trigger the gate mechanism at the right time. Based on sensor data, the servo motor responded promptly by opening and closing the gate automatically, ensuring smooth vehicle entry and exit without manual intervention.

The LED indicators provided a clear and immediate visual representation of parking availability. A green light indicated that the parking slot was free, while a red light signaled that it was occupied. This simple visual system proved to be highly effective for users, allowing them to quickly identify available spaces.

In addition, the buzzer functioned as an alert mechanism, activating when a vehicle came too close to a restricted area or sensor. This added an extra layer of safety and helped prevent potential collisions or improper parking.

From an IoT perspective, the system successfully established communication with the PubNub cloud platform, enabling real-time data transmission and monitoring. Sensor data was continuously updated on the cloud, allowing remote access to parking information. Furthermore, remote control commands were tested and executed successfully, demonstrating the system's capability for remote operation and control, which is a key feature of modern IoT applications.

Overall, the system performed reliably during testing and demonstrated strong potential as a smart parking solution. The integration of sensors, actuators, and cloud communication highlights the effectiveness of IoT technology in improving parking management, reducing manual effort, and enhancing user convenience.

CHAPTER 7: LIMITATIONS AND FUTURE SCOPE

7.1 Limitations

Although the proposed Smart Parking System performs effectively under controlled conditions, there are several limitations that need to be considered.

Firstly, the current implementation supports only a limited number of parking slots (or a single slot in basic prototype). This restricts its practical usability in large-scale environments such as shopping malls or public parking areas where multiple slots need to be monitored simultaneously. Expanding the system would require additional sensors and more complex data handling. Secondly, the accuracy of ultrasonic sensors can be influenced by environmental factors such as dust, temperature variations, uneven surfaces, or obstacles near the sensor. These factors may lead to incorrect detection of vehicle presence, affecting the reliability of the system.

Another limitation is the system's dependency on a stable internet connection for cloud communication. Since real-time data updates and remote monitoring rely on cloud platforms like PubNub or Azure, any network interruption may result in delays or loss of data transmission. Additionally, the system currently lacks a dedicated user interface or mobile application, which limits user interaction.

Furthermore, the prototype is designed using basic hardware components, which may not be fully optimized for long-term deployment in outdoor environments. Factors such as weather conditions, power fluctuations, and hardware durability can affect system performance.

7.2 Future Scope

The Smart Parking System can be significantly enhanced in the future to make it more efficient, scalable, and suitable for real-world applications.

One of the major improvements would be to extend the system to support multiple parking slots by integrating additional sensors and developing a centralized monitoring system. This would allow the system to be deployed in large areas such as malls, airports, and smart cities. Another important enhancement is the development of a mobile application, where users can view real-time parking availability, navigate to free slots, and receive notifications. This would greatly improve user experience and accessibility.

The system can also be upgraded by integrating online reservation and digital payment features, allowing users to book parking spaces in advance and make payments through mobile platforms. This would reduce congestion and waiting time.

In addition, advanced technologies such as Artificial Intelligence (AI) and Machine Learning (ML) can be incorporated to predict parking availability based on historical data and usage patterns. This would enable smarter decision-making and better traffic management. Further improvements may include the integration of license plate recognition systems, which can automate vehicle identification and enhance security. The system can also be connected with smart city infrastructure, enabling seamless communication between traffic systems, parking systems, and navigation services.

Finally, the system can be made more robust by improving hardware design for outdoor deployment, incorporating weather-resistant components, backup power systems, and more reliable communication technologies.

CHAPTER 8 REFERENCES

- [1] Arduino, "Arduino Documentation," [Online]. Available: <https://www.arduino.cc>
- [2] PubNub, "Realtime IoT Platform," [Online]. Available: <https://www.pubnub.com>
- [3] Kaggle, "Parking Occupancy Dataset," [Online].
- [4] IoT Smart Parking Research Papers, IEEE